SCHEDORA

SaaS Appointment Scheduling and Calendar Management Platform

Complete Project Guidebook and Technical Bible

Product Roadmaps | Database Schemas | API Workflows | System Architectures

Author:	Technical Product Manager and Architect Team
Target Audience:	Product Demos, Portfolio, Technical Reviewers
Document Version:	1.0.0 (Production Grade)
Release Date:	May 2026

Table of Contents

1. Project Overview and Core Vision

Schedora is an enterprise-ready, premium SaaS appointment scheduling and calendar management platform engineered to eliminate the inefficiencies of modern business coordination. Drawing inspiration from leading products like Calendly, Linear, and Stripe, Schedora offers a highly optimized, dual-theme interface that allows professionals, remote teams, and agency owners to manage availability, automate appointment bookings, synchronize multiple external calendars, collect payments for scheduled slots, and gather detailed engagement analytics.

Vision and Mission

The vision of Schedora is to build a unified layer for human availability. Our mission is to return time to professionals by transforming scheduling from an active cognitive chore of email back-and-forth into a passive, frictionless, background utility. We achieve this by combining robust multi-calendar conflict checks, declarative availability rules, and secure billing logic within an intuitive dashboard experience.

Product Uniqueness

- **Modern Visual Design System:** Features a highly refined glassmorphic dashboard optimized for both light and dark aesthetics, adhering to strict WCAG contrast ratios.
- **Secure Stripe Elements Integration:** Direct embedded checkouts utilizing Stripe's official UI primitives, allowing seamless monetization of paid consultation hours directly on event pages.
- **Multi-Tenant Tenant Isolation:** Configured using PostgreSQL Row Level Security (RLS) policies inside Supabase to enforce data boundaries.

2. Problem Statement and Solutions

Traditional scheduling relies on manual email correspondence, exposing businesses to timezone conversion errors, double bookings, and disjointed team calendars. Schedora directly addresses these organizational pain points:

- **Manual Scheduling Loop:** Studies show that booking a single meeting manually requires an average of 4.8 emails. Schedora reduces this cost to a single link share, resolving scheduling conflicts automatically.
- **Double Bookings and Fragments:** Professionals work across multiple personal and work calendars. Schedora aggregates integrated calendar accounts, scanning them in real time for blockades before presenting open availability.
- **Timezone Confusion:** Meetings spanning multiple continents invite errors. Schedora automatically captures the visitor's local system timezone, cross-references it with the host's operating settings, and converts all intervals correctly.
- **Paid Consultations Overheads:** Setting up invoices before booking consultation calls leads to dropoffs. Schedora embeds Stripe Elements directly into the checkout form, verifying payment details synchronously prior to booking confirmation.

3. Product Idea and Concept Evolution

The Schedora concept originated from analyzing the friction in freelance and agency consultation client-onboarding pipelines. Existing solutions either redirected users to external landing pages that disrupted branding, or lacked direct, native payment integrations that validated calendar availability prior to capture. Schedora was designed from the ground up as a developer-friendly scheduling engine and premium dashboard.

The design system evolved from a simple calendar slot generator into a multi-tenant SaaS workspace. During this evolution, visual responsiveness was identified as a core metric for user conversion. Consequently, we integrated a global dark-mode/light-mode theme framework powered by Tailwind v4 CSS variables and custom transitions, allowing the dashboard to automatically scale from personal use to enterprise environments.

4. Feature Breakdown and Details

Schedora breaks down into the following core product modules, each engineered with custom database schemas and user workflows:

- **Dashboard and Control Panel** The user's central control cockpit. Exposes a visual overview of scheduling metrics, current booking tallies, active event templates, and quick actions to copy booking links or add custom availability.
- **Event Types Engine** Enables declarative scheduling templates. Users define duration, meeting platform (Google Meet, Custom link), visual colors, buffers, dynamic slugs, and whether slots are free or paid.
- **Bookings Tracker** Renders lists of upcoming, past, and cancelled appointments. Attendees can view specific appointment details, access meeting links, and process self-service cancellations according to host rules.
- **Dynamic Availability Management** Granular day-of-week settings mapping available business hours. Supports buffer times (e.g., 10-minute breaks before/after meetings) and daily booking limits to protect the host's calendar.
- **Team Workspace and Collaboration** Allows organization managers to create team workspaces, invite members via secure cryptographic tokens, and distribute admin or member roles for team-based scheduling.
- **Embedded Stripe Billing** Supports premium billing plans (Pro/Enterprise) using Stripe's official SDK. Integrates custom sandbox checks and live payment intents directly into the app checkout modals.
- **Dynamic Theme Engine** A top-right visual header switch changing the entire dashboard from deep dark space purple to linear light white. Features clean, smooth page transitions and theme-aware SVG charts.

5. Step-by-Step System Workflows

To understand the core coordination engine of Schedora, we map the step-by-step logic of the four primary workflows powering the SaaS application.

A. The Booking Workflow

- 1. Host shares booking link (e.g., /book/username/30-min-consult).
- 2. Visitor opens the page; client requests host event configuration and availability settings from Database.
- 3. Availability Engine reads integrated Google/Outlook calendars for real-time conflict blocks.
- 4. Visitor selects an open date and time slot, and fills in attendee details (Name, Email, Notes).
- 5. (Optional) If it is a paid event, the Stripe Elements modal opens and processes card authorization synchronously.
- 6. Server checks slot availability again (concurrency check) and writes booking record to database.
- 7. Integration Engine creates an external event on the host's connected calendar and generates a Google Meet link.
- 8. Dynamic email/in-app notifications are dispatched to both the host and visitor with calendar attachments.

B. Stripe Subscription and Billing Workflow

- 1. User accesses the Billing Tab in Settings and selects the Pro or Enterprise Plan.
- 2. The Schedora client initiates a request to POST /api/billing/create-subscription.
- 3. Express backend contacts Stripe, creates/retrieves Stripe Customer, and builds a Subscription with payment intent.
- 4. Client receives the client_secret and mounts Stripe Card Elements inside the secure modal.
- 5. Stripe SDK securely transmits card data, performs 3D-Secure check, and completes payment.
- 6. Stripe Webhook listener processes customer.subscription.created and updates subscription status in Supabase database.
- 7. PlanEnforcer component refreshes UI to unlock premium tier features (unlimited event types, team invites).

C. Calendar Authorization and Sync Workflow

- 1. User clicks 'Connect Calendar' in integrations dashboard.
- 2. Client redirects user to Google/Microsoft OAuth page requesting read/write offline scopes.
- 3. User approves request; provider returns authorization code to Schedora server redirect URI.
-

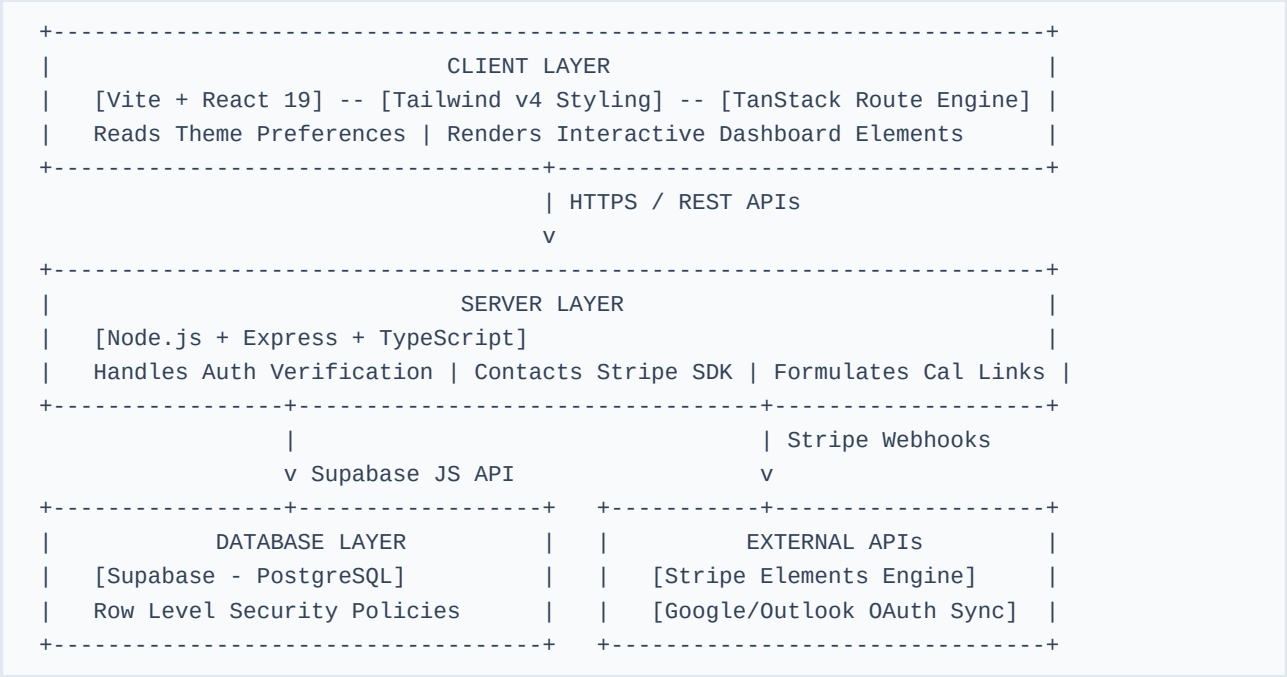
4. Schedora Server exchanges authorization code for access_token and refresh_token, saving it to integrations table.

- 5. Background synchronization cron task polls events periodically, translating them into calendar_events records.

6. Technical Architecture and Layers

Schedora is architected as a modern, split-layer Single Page Application (SPA) communicating with a secure server and database backend. Multi-tenancy isolation is enforced at the database level.

Schedora High-Level System Architecture



Architecture Details

- **Frontend Client Layer:** Constructed using React 19, Vite, and TanStack Router. State is shared via context and local hooks. Tailwind v4 is used for style sheets, mapping color variables directly to system state classes.
- **Backend Server Layer:** An Express.js application written in TypeScript (run via tsx). It exposes API endpoints for Stripe Elements creation, webhook event verification, and calendar authorization flows.
- **Database and Security Layer:** Supabase PostgreSQL holds relational tables. All requests run through PostgreSQL RLS policies to prevent Cross-Tenant data access.

7. Technology Stack Selection

The technologies chosen for Schedora represent modern web development standards. We selected each framework based on scalability, developer velocity, and performance metrics.

Technology	Component / Layer	Selection Rationale
React 19 & Vite	Frontend Application	Provides fast HMR, declarative component updates, and clean hooks usage for managing interactive visual
TailwindCSS v4	Visual Interface / CSS	Utilizes a compile-time CSS-only theme engine. CSS variables allow dynamic theme toggles with zero runtime layout calculations.
TanStack Router	Routing / Navigation	Typesafe route structure ensuring compile-time safety. Smooth transitions between child route slots without layout
Express.js + tsx	Backend API Server	Lightweight, highly performant Node.js runtime. Provides middleware routing for Stripe SDK and Google API synchronization.
Supabase / Postgres	Database / Storage	PostgreSQL database with built-in RLS policies. Trigger hooks automate profile and subscription initialization during Auth signup.
Stripe Elements	Payment Gateway Layer	Allows PCI-compliant embedded checkout forms inside Schedora dialog boxes, reducing payment conversion drops.
Lucide React	UI Component Icons	Consistent vector icons with built-in SVG path scaling, simplifying clean rotate/scale micro-animations on theme buttons.

8. Database Design and Relational Schema

Schedora uses a fully relational database schema designed for efficient querying, cascading deletion, and strict data security through Row Level Security (RLS) policies. Below is the technical data dictionary.

Table: public.profiles

Column	Data Type	Constraints	Description
id	UUID	PRIMARY KEY, REFERENCES	Unique user authentication identifier
username	TEXT	UNIQUE	User custom handle for booking links
email	TEXT	NOT NULL	Primary user email address
timezone	TEXT	DEFAULT 'UTC'	Operating timezone for booking calculations
created_at	TIMESTAMPTZ	DEFAULT now()	Timestamp when profile was created

Table: public.event_types

Column	Data Type	Constraints	Description
id	UUID	PRIMARY KEY	Unique template identifier
user_id	UUID	REFERENCES public.profiles(id)	Owner profile relationship key
title	TEXT	NOT NULL	Name of the booking template
duration	INTEGER	DEFAULT 30	Meeting slot length in minutes
slug	TEXT	NOT NULL	URL route key (scoped per
is_active	BOOLEAN	DEFAULT true	Toggle to enable or disable public booking
buffer_time	INTEGER	DEFAULT 0	Inter-meeting buffer offset in minutes

Table: public.bookings

Column	Data Type	Constraints	Description
id	UUID	PRIMARY KEY	Unique booking reference ID
user_id	UUID	REFERENCES public.profiles(id)	Assigned host profile key
attendee_name	TEXT	NOT NULL	Full name of the scheduling
attendee_email	TEXT	NOT NULL	Email address of the visitor
booking_date	TEXT	NOT NULL	Target scheduling date string (YYYY-MM-DD)
booking_time	TEXT	NOT NULL	Target start time slot string (HH:MM)
status	TEXT	CHECK status IN upcoming/past/cancelled	Current booking lifecycle state
meeting_link	TEXT	NULLABLE	Generated video meeting endpoint

Table: public.subscriptions

Column	Data Type	Constraints	Description
id	UUID	PRIMARY KEY	Unique billing record ID
user_id	UUID	UNIQUE, REFERENCES profiles	Assigned subscriber key
plan	TEXT	CHECK	Legacy tier mapping name
subscription_plan	TEXT	DEFAULT 'free'	Active Stripe billing product tier
stripe_customer_id	TEXT	NULLABLE	Stripe customer reference token
stripe_subscription_id	TEXT	NULLABLE	Stripe active billing reference
subscription_end	TIMESTAMPTZ	NULLABLE	Current billing cycle expiry timestamp

Database Relationships and Triggers

- **Cascade Deletion:** All tables referencing public.profiles (e.g. event_types, bookings, availability_settings, subscriptions) use ON DELETE CASCADE to ensure proper database cleanup upon account termination.
- **Auth Trigger Functions:** Upon successful user creation in Supabase auth.users, database triggers public.handle_new_user(), public.handle_new_user_subscription(), and public.handle_new_user_availability() automatically populate default settings, a free subscription record, and baseline weekday business hours (9:00 AM - 5:00 PM, Mon-Fri).

9. UI/UX Design System and Theme Engine

Schedora's user experience is centered on premium SaaS design guidelines. The styling system leverages Tailwind v4 theme properties, combining dark mode elegance with clean light mode aesthetics.

- **The Theme Toggle Switch:** Positioned globally in the header. Employs SVG rotation and scaling animations using Tailwind classes (transition-all duration-300). Clicking the button smoothly transitions the background and borders over a 300ms window without layouts shifting.
- **Light Mode Philosophy:** Inspired by Stripe's dashboard structure. Replaces deep dark backgrounds with crisp slate colors (#f8fafc), white cards, and subtle drop shadows (box-shadow: 0 1px 2px rgba(0,0,0,.04), 0 8px 24px rgba(15,23,42,.06)) to establish visual hierarchy.
- **Dark Mode Philosophy:** A dark cosmic aesthetic (#09090b) with deep indigo cards (#0a0a0f) and purple branding elements. Ambient glowing borders replace drop shadows, maintaining contrast and reducing eye strain.
- **Typography Hierarchy:** Utilizes the Inter font family. It implements clear type scales, ranging from small tracking labels (text-xs tracking-wide uppercase) to bold primary titles (text-2xl font-bold).

10. Implementation Guide and Code Architecture

Let us examine Schedora's technical implementations. The scheduler relies on two major algorithmic systems:

A. Realtime Overlap and Conflict Check Algorithm

To prevent double bookings, the server merges availability settings with booked slots and active external calendar events. A target booking slot (StartA, EndA) conflicts with an existing slot (StartB, EndB) if:

Overlap Logic Formula

```
StartA < EndB AND StartB < EndA
```

In the backend scheduling route, this check is executed as a relational query inside PostgreSQL, scanning bookings table and connecting to external OAuth feeds in parallel to verify the slot is free before committing the transaction.

B. Multi-Mode Stripe Elements Controller

To enable paid bookings, Stripe Elements is mounted in a dialog modal. The React client dynamically checks whether the Stripe keys are configured. If missing or sandbox mode is active, it falls back to a sandbox simulator which mocks card scenarios. Below is a code block showing the backend webhook controller signature.

Stripe Webhook Listener Controller

```
app.post('/api/billing/webhook', express.raw({ type: 'application/json' }),
  async (req, res) => {
```

```
    const sig = req.headers['stripe-signature'];
    let event;
    try {
      event = stripe.webhooks.constructEvent(req.body, sig, WEBHOOK_SECRET);
    } catch (err) {
      return res.status(400).send(`Webhook Error: ${err.message}`);
    }
    if (event.type === 'customer.subscription.updated') {
      const sub = event.data.object;
      await updateDbSubscription(sub.customer, sub.status, sub.current_period_end);
    }
    res.json({ received: true });
  }
);
```

11. Business Model and Subscription Plans

Schedora operates a product-led growth (PLG) freemium business model designed to attract single users while monetizing power scheduling features and collaborative teams.

Plan Name	Price / Month	Target Audience	Included Capabilities
Free Tier	$0	Freelancers / Solo users	1 active event type slug, standard email confirmations, 30-day booking histories, and UTC timezone calculations.
Pro Tier	$15	Consultants / Independent	Unlimited active event types, Stripe-paid event checkouts, automated follow-up buffer setups, and CRM connections.
Enterprise Tier	$49	Teams / Agencies	Multi-member team workspaces, round-robin scheduling queues, collective availability blocks, and custom branding logs.

12. Security System and Scalability Roadmap

To scale to enterprise volumes, Schedora implements strict layers of technical defense and performance optimizations.

A. Authentication Security and Row Level Security (RLS)

Client requests communicate directly with Supabase via JSON Web Tokens (JWT). The PostgreSQL database implements row level security (RLS) policies. For instance, the policy governing event type configuration reads:

PostgreSQL Row Level Security Policy

```
CREATE POLICY "Users can manage own event types"
ON public.event_types FOR ALL
USING (auth.uid() = user_id);
```

This ensures that even if an attacker attempts to fetch event configs of another user via API parameters, the PostgreSQL execution engine filters the dataset, preventing unauthorized access.

B. Scalability Roadmap

- **Database Indexing:** Indexes on user_id, booking_date, and event_slug ensure rapid availability checks, scaling to millions of entries.
- **Realtime Polling Offloading:** Transitioning from client-side cron polling to webhook-based event updates via Google PubSub and Microsoft Event Hubs.
- **Edge Cache Distribution:** Deploying host booking forms on Vercel Edge networks to resolve and serve slots within 50ms globally.

13. Future Roadmap and AI Integrations

Schedora's engineering roadmap incorporates cutting-edge automation and integration capabilities to secure a long-term competitive advantage in the SaaS scheduling market.

- **AI Scheduling Assistant:** A natural-language scheduling interface (e.g., 'Book a 45-minute sync with Sarah next Tuesday afternoon'). The AI agent analyzes conversational emails, resolves host availability parameters, and books the event automatically.
- **Smart Availability Suggestions:** An optimization engine that analyzes past meeting outcomes, cancellation frequencies, and peak concentration hours to suggest optimal meeting slots, reducing host cognitive fatigue.
- **Native Video Integrations:** Direct API authorization paths for Zoom, Microsoft Teams, and Webex, generating secure dynamic credentials for every meeting booked.
- **CRM Sync Connections:** Auto-syncing scheduled contact profiles and conversation histories directly to Hubspot, Salesforce, and Zoho CRM pipelines.

14. Competitive Analysis

A comparison of Schedora with market leaders reveals how Schedora captures the modern SaaS developer and entrepreneur audience.

Metric	Schedora	Calendly	Motion
Target Audience	Freelancers / Small	Corporate Sales / HR	Productivity Seekers
Base Price	Free / Pro ($15) / Team ($49)	Free / Custom ($12 - $20)	$34 (No free tier)
Payment Flow	Stripe Elements Embedded	Stripe Redirection Link	Not natively supported
Theme Customizer	Dynamic Light / Dark Toggle	Basic light layout	Fixed interface layout
Data Privacy	Multi-Tenant isolation via RLS	Proprietary database	Proprietary cloud storage

15. Stakeholder Product Demo Script

This presentation script is structured to demonstrate the value of Schedora to hackathon judges, investors, and technical stakeholders.

Schedora Live Product Demo Script

```
[0:00 - Introduction]
"Good morning. Let's look at a common business frustration: scheduling. We waste hours in back-and-fort

[0:45 - The Host Dashboard]
"Here is Schedora's host control panel. You are looking at a dashboard that supports both clean light t

[1:30 - The Booking Flow and Stripe Elements]
"When I share this booking link with a client, they see my real-time availability in their local timezo

[2:15 - Analytics and Team Management]
"Returning to Schedora, we can track our booking trends and event breakdown. If I manage an agency, I c

[2:45 - Architecture and Conclusion]
"Under the hood, Schedora runs on React, Node.js, and Supabase. Tenant isolation is enforced in the dat
```

16. Structured Elevator Pitches

Pitch frameworks optimized for different time constraints, designed for networking events, investors, or technical interviews.

A. The 30-Second Elevator Pitch

"Schedora is a premium SaaS scheduling platform that helps professionals automate bookings and monetize consultations. Unlike old tools that redirect users to third-party billing pages, Schedora uses secure, embedded Stripe checkouts and real-time calendar syncing inside a responsive design. It simplifies meeting scheduling for professionals and teams."

B. The 60-Second Investor Pitch

"Scheduling meetings is a major administrative time sink. Schedora solves this by providing a unified availability engine. Hosts configure booking templates and link their external calendars. Attendees select times in their local timezones and pay for sessions directly through an embedded checkout. By utilizing Supabase's multi-tenant database isolation, Schedora offers enterprise-grade security on a flexible tech stack. We are positioning Schedora to capture the growing freelance and remote agency market, offering team scheduling at a fraction of the cost of legacy platforms. Schedora makes scheduling efficient, secure, and beautiful."

C. The 3-Minute Presentation Explanation

- **Introduce the Coordination Gap:** Businesses struggle with manual coordination across fragmented tools. This leads to timezone confusion, double bookings, and payment friction.
- **Demonstrate the Solution:** Schedora bridges this gap with an intuitive interface, deep calendar syncing, and embedded payment options, ensuring hosts get paid before sessions are booked.
- **Highlight Technical Integrity:** Built on React, Node, and Supabase. The Postgres database implements Row Level Security (RLS) for data protection. It supports light and dark themes with transitions for a modern SaaS feel.
- **Map out the Business Potential:** A freemium model that scales to team workspaces, with an API-first approach that lays the groundwork for AI-powered scheduling.

17. Interview Q&A Section

This section prepares candidates to explain Schedora's technical design and implementation choices during engineering interviews.

Q: Tell me about Schedora. What is it and why did you build it?

A: Schedora is a SaaS scheduling platform built to simplify calendar management and paid consultations. I built it to solve coordination problems like timezone conversions and manual scheduling back-and-forths, while keeping the checkout flow entirely in-app using Stripe Elements.

Q: Why did you choose Supabase and PostgreSQL instead of a NoSQL database?

A: Scheduling data is relational: bookings link to users and event templates, availability settings belong to profiles, and teams manage members. Relational schemas enforce integrity. PostgreSQL allows us to run overlap checks using date ranges and enforce security rules using Row Level Security (RLS) directly in the database.

Q: How did you implement Stripe Elements? What happens when a checkout completes?

A: I integrated Stripe Elements using the official SDK. On selection of a plan or paid slot, the client requests a payment intent secret from the Express server. The client mounts the elements inside a modal. When the card is approved, the Stripe webhook listener sends a customer.subscription.updated event to update the database state, unlocking premium tier capabilities.

Q: How does the application handle timezones dynamically?

A: The frontend uses the browser's Intl API to detect the visitor's local timezone. The host's operating settings are stored in UTC. When the booking page loads, availability times are parsed, checked for calendar conflicts, and converted to the attendee's timezone for booking, before writing back to the database in UTC.

18. Technical Challenges and Solutions

Engineering a robust scheduling engine presented several technical challenges. Below are three key obstacles we overcame during development.

1. Dynamic Calendar Synchronization and API Rate Limits

Challenge: Querying the Google Calendar API on every booking page load caused slow response times and rate-limiting issues.
Solution: We built a sync layer that caches calendar blocks in the calendar_events table. A background worker syncs external changes, while a webhook listener updates local records when calendar events are modified, reducing external API dependencies.

2. Race Conditions on Time Slot Bookings

Challenge: Two visitors could select and book the exact same time slot simultaneously, causing double bookings.
Solution: We implemented a database transaction check at the API level. Before saving a booking, the server verifies the slot remains available. The event type user_id and slug are constrained by a UNIQUE database index, ensuring only one booking per slot is processed.

3. Smooth Styling Transitions with Tailwind v4

Challenge: Switching themes caused visual flashes on complex elements like sidebars and card shadows.
Solution: We configured Tailwind v4 theme variables to map to CSS variables in styles.css. By applying transition properties (background-color, border-color, box-shadow) with a 300ms ease, we ensured smooth transitions across the dashboard.

19. Portfolio and Resume Summaries

Copy-pasteable descriptions optimized for professional networking platforms and resume applications.

A. Resume Bullet Points

- **Built a multi-tenant SaaS scheduling platform using React, Node.js, and Supabase,** implementin g Row Level Security (RLS) policies to protect user data.
- **Integrated Stripe Elements for native checkouts,** building a backend webhook listener to sync subscription statuses and manage plans.
- **Designed a responsive design system with a Light/Dark theme toggle,** using Tailwind v4 variables to ensure smooth transitions across components.
- **Created an availability engine that syncs with external calendars,** performing conflict checks in PostgreSQL to prevent double bookings.

B. LinkedIn Project Description

Schedora is a SaaS scheduling platform built to simplify availability management and paid consultations. It features a clean visual interface with light and dark mode toggles, real-time calendar syncing, and direct Stripe Elements integrations. Built on a modern stack of React, Vite, Node.js, and Supabase (PostgreSQL), the application implements multi-tenant isolation via database Row Level Security (RLS). Schedora helps professionals organize their availability, manage bookings, and automate meeting workflows.

20. Conclusion and Future Outlook

Schedora demonstrates how a modern technical stack can solve scheduling challenges simply and securely. By combining React, Node.js, and Supabase, the platform provides a fast scheduling engine and dashboard experience.

Building Schedora highlighted the value of structured relational database designs, secure third-party checkout flows, and clean, modern frontends. As the platform grows, integrating AI scheduling assistants and smart availability options will further reduce scheduling friction for busy professionals. Ultimately, Schedora shows that automated coordination can be reliable, secure, and easy to use.

END OF DOCUMENT